编写一个基本的 Linux shell

是一个有趣而且有挑战性的任务。下面是一个简单的示例，说明如何编写一个基本的 shell，该示例仅支持最基本的命令解释和执行，并使用了创建子进程的方式执行命令。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_COMMAND_LENGTH 100
#define MAX_ARGUMENTS 10

void execute_command(char *command) {
    char *args[MAX_ARGUMENTS];
    int i = 0;

    // 将命令分割成参数
    char *token = strtok(command, " ");
    while (token != NULL && i < MAX_ARGUMENTS - 1) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }
    args[i] = NULL; // 参数列表以 NULL 结尾

    // 创建子进程执行命令
    pid_t pid = fork();
    if (pid < 0) {
        fprintf(stderr, "Fork failed\n");
        exit(EXIT_FAILURE);
    } else if (pid == 0) {
        // 子进程执行命令
        if (execvp(args[0], args) == -1) {
            perror("Execution failed");
            exit(EXIT_FAILURE);
        }
    } else {
        // 父进程等待子进程执行完毕
        waitpid(pid, NULL, 0);
    }
}

int main() {
    char command[MAX_COMMAND_LENGTH];

    while (1) {
        // 打印提示符
        printf("Shell> ");
```

```
        fflush(stdout);

        // 读取命令
        if (fgets(command, sizeof(command), stdin) == NULL) {
            break;
        }

        // 删除末尾的换行符
        command[strcspn(command, "\n")] = '\0';

        // 如果输入是 "exit" 则退出 shell
        if (strcmp(command, "exit") == 0) {
            break;
        }

        // 执行命令
        execute_command(command);
    }

    return 0;
}
```

这个简单的示例展示了一个非常基本的 shell 实现。它接受用户的输入，解析命令和参数，然后创建子进程来执行命令。父进程在等待子进程执行完毕后继续接收下一个命令。这是一个最基本的示例，真正的 shell 还有许多功能和优化可以添加，比如处理内置命令、I/O 重定向、管道等。

当编写一个命令解释程序（即 shell）时，通常需要

实现一系列步骤

来接收、解释和执行命令。以下是对每个步骤的简要解释：

1. 接收命令：
    - Shell 需要从用户输入中获取命令。这可以通过不同的方式实现，比如使用 `fgets` 函数从标准输入读取用户的命令。
2. 解释命令：
    - 接收到命令后，Shell 需要将命令解析成可执行的格式。这包括将命令字符串分解成命令和参数，并进行必要的处理，比如删除末尾的换行符等。
3. 执行命令：
    - 解释完命令后，Shell 需要执行相应的命令。这通常涉及创建子进程来执行命令。在 Linux 中，可以使用 `fork` 函数创建子进程，并使用 `exec` 系列函数在子进程中执行命令。
4. 创建子进程：
    - 通过调用 `fork` 函数来创建子进程。`fork` 函数在父进程中返回子进程的进程 ID，而在子进程中返回 0。父进程可以通过检查返回值来确定它是在父进程还是在子进程中运行。
5. 执行一个程序以及父进程如何继续子进程的工作：

- 在子进程中，可以使用 `exec` 系列函数来执行命令。这些函数会将当前进程替换为新的程序，从而执行相应的命令。在父进程中，通常需要等待子进程执行完毕。可以使用 `wait` 或 `waitpid` 函数来等待子进程的结束。父进程可以通过这些函数来阻塞自己，直到子进程退出，并且可以获取子进程的退出状态。

综上所述，命令解释程序（shell）的主要流程包括接收命令、解释命令、创建子进程执行命令，并在父进程中继续等待子进程的工作。

接收命令的实现

涉及从用户输入中读取命令字符串。在这个示例中，我们使用 `fgets` 函数从标准输入读取用户的命令。下面是对接收命令的实现步骤的详细解释：

1. 在 `main` 函数中，我们定义了一个字符数组 `command[MAX_COMMAND_LENGTH]`，用于存储用户输入的命令。

```
char command[MAX_COMMAND_LENGTH];
```

2. 在 `while` 循环中，我们重复以下步骤以接收用户的命令：

```
while (1) {
    // 打印提示符
    printf("Shell> ");
    fflush(stdout);

    // 读取命令
    if (fgets(command, sizeof(command), stdin) == NULL) {
        break;
    }

    // 删除末尾的换行符
    command[strcspn(command, "\n")] = '\0';

    // 如果输入是 "exit" 则退出 shell
    if (strcmp(command, "exit") == 0) {
        break;
    }

    // 执行命令
    execute_command(command);
}
```

3. 在循环中，我们首先打印一个提示符，提示用户输入命令。这里我们使用 `printf` 函数打印 `"Shell> "`。

```
printf("Shell> ");
fflush(stdout);
```

4. 接着，我们使用 `fgets` 函数从标准输入读取用户输入的命令，并将其存储在 `command` 数组中。

```
if (fgets(command, sizeof(command), stdin) == NULL) {
    break;
}
```

`fgets` 函数从标准输入读取一行字符，并将其存储在 `command` 数组中。第二个参数 `sizeof(command)` 指定了存储命令的缓冲区大小，以避免缓冲区溢出。如果读取失败（比如用户输入了文件结束符，通常是 `Ctrl+D`），则退出循环。

5. 在读取用户输入的命令后，我们使用 `strcspn` 函数来查找并删除命令字符串中的换行符。这是因为 `fgets` 函数会将包括换行符在内的整行字符读取到缓冲区中。

```
command[strcspn(command, "\n")] = '\0';
```

`strcspn` 函数返回指定字符在字符串中第一次出现的位置，我们使用它找到换行符的位置，并将其替换为字符串结束符 `'\0'`，从而将换行符删除。

6. 最后，我们检查用户输入的命令是否为 "exit"，如果是则退出循环，结束程序的执行。

```
if (strcmp(command, "exit") == 0) {
    break;
}
```

如果用户输入的命令不是 "exit"，则调用 `execute_command` 函数来解释并执行用户输入的命令。

解释命令的实现

涉及将用户输入的命令字符串解析为可执行的命令及其参数列表。在这个示例中，我们使用 `strtok` 函数将命令字符串分割成命令和参数，并将它们存储在一个参数数组中。下面是对解释命令的实现步骤的详细解释：

1. 在 `execute_command` 函数中，我们首先定义了一个参数数组 `args[MAX_ARGUMENTS]`，用于存储命令及其参数。

```
char *args[MAX_ARGUMENTS];
```

2. 接着，我们使用 `strtok` 函数将命令字符串分割成命令及其参数。`strtok` 函数是一个字符串分割函数，它将字符串分割成一系列子字符串，其返回值为分割后的第一个子字符串的指针。

```
char *token = strtok(command, " ");
while (token != NULL && i < MAX_ARGUMENTS - 1) {
    args[i++] = token;
    token = strtok(NULL, " ");
}
args[i] = NULL; // 参数列表以 NULL 结尾
```

在上述代码中，我们首先使用 `strtok(command, " ")` 将命令字符串 `command` 按照空格分割成第一个子字符串（即命令），并将其赋值给 `token`。然后，我们使用一个 `while` 循环来依次将分割得到的参数存储在 `args` 数组中。在循环中，我们通过 `strtok(NULL, " ")` 来获取下一个分割后的子字符串（即参数），并将其存储在 `args` 数组中。循环终止条件包括分割结束或者已达到最大参数数量。最后，我们将参数列表以 `NULL` 结尾，以便后续的命令执行函数正确识别参数列表的结束。

3. 当解释命令完成后，`args` 数组中存储了命令及其参数，可以作为参数传递给 `execvp` 函数执行命令。

执行命令的实现

涉及在创建的子进程中调用适当的系统调用来执行用户输入的命令。在这个示例中，我们使用 `execvp` 函数来执行命令，该函数会在子进程中搜索系统 `PATH` 变量指定的路径以找到要执行的程序。下面是对执行命令的实现步骤的详细解释：

1. 在 `execute_command` 函数中，我们首先创建了一个子进程，以便在子进程中执行用户输入的命令。

```
pid_t pid = fork();
if (pid < 0) {
    fprintf(stderr, "Fork failed\n");
    exit(EXIT_FAILURE);
} else if (pid == 0) {
    // 子进程执行命令
    // ...
} else {
    // 父进程等待子进程执行完毕
    // ...
}
```

在上述代码中，我们使用 `fork` 函数创建了一个新的进程。如果 `fork` 函数返回值小于 0，则表示创建子进程失败，我们会输出错误信息并退出程序。如果返回值等于 0，则表示当前代码在子进程中执行。如果返回值大于 0，则表示当前代码在父进程中执行，且返回值为子进程的进程 ID。

2. 在子进程中，我们调用 `execvp` 函数来执行用户输入的命令。`execvp` 函数会搜索系统 `PATH` 变量指定的路径以找到要执行的程序，并使用传入的参数列表来执行该程序。

```
if (execvp(args[0], args) == -1) {
    perror("Execution failed");
    exit(EXIT_FAILURE);
}
```

在上述代码中，`args[0]` 表示命令，而 `args` 数组则是参数列表。如果 `execvp` 函数执行失败，它会返回 -1，并且会设置 `errno` 变量来指示失败的原因。我们使用 `perror` 函数来打印错误信息，并退出子进程。

3. 在父进程中，我们使用 `waitpid` 函数等待子进程执行完毕。`waitpid` 函数会阻塞父进程，直到指定的子进程结束。在这里，我们传入 `pid` 作为参数，表示等待任何一个具有指定进程 ID 的子进程结束。

```
waitpid(pid, NULL, 0);
```

这样，当子进程执行完毕后，父进程会继续执行，并且会从 `waitpid` 函数中返回。